

# Getting Started

BuMoChi

## Overview

This guide introduces BuMoChi through a sequence of small tests. Perform them in order. Each step reuses the working pipeline and material created by the previous step.

If the software is not installed yet, follow [Installation](#). To set up the system to receive animation data and run animations on Godot, see: [Setup](#).

The first five exercises use the current default configuration:

| Component                      | Setting                    |
|--------------------------------|----------------------------|
| XR-Animator VMC output         | <b>127.0.0.1:39537</b>     |
| <b>BunrakuOSCEncoder</b> input | <b>39537</b>               |
| Bmc input                      | <b>57130</b>               |
| <b>BunrakuOSCDecoder</b> input | <b>39538</b>               |
| Ishidomaru Godot VMC input     | <b>39539</b>               |
| Godot project                  | <b>Seed_2_Ishidomaru_C</b> |

Run terminal commands from the root of the BuMoChi repository. Evaluate SuperCollider code one parenthesized block or one statement at a time.

## Prepare the local test pipeline

This preparation omits OSCGroups so that the first tests remain local and easy to diagnose. For the complete pipeline procedure, see [Setup](#).

## Animate the Ishidomaru avatar from XR-Animator

Move in front of the webcam. Ishidomaru should follow the movement in Godot.

Confirm in the Bmc monitor that the dynamic **received** count is increasing. The encoder terminal should also report received VMC data and frames sent to Bmc.

If the avatar does not move, check the pipeline from downstream to upstream:

1. Godot is running **Seed\_2\_Ishidomaru\_C** and listening on **39539**.
2. The decoder is listening on **39538** and allows target port **39539**.
3. Bmc is listening on **57130** and forwarding to the decoder.
4. The encoder is listening on **39537** and sending to Bmc on **57130**.
5. XR-Animator is sending VMC to **127.0.0.1:39537**.

For deeper diagnosis, see [Port Number Setup](#).

## Animate the Ishidomaru avatar from an algorithmic clip

This example captures the latest valid live pose as a model-compatible reference, then generates a four-second clip in which the hips move gently from side to side. Using a valid received pose preserves Ishidomaru's bone proportions and VMC coordinate conventions.

First move into a comfortable upright pose and confirm that Ishidomaru is visible. Then evaluate:

```

(
var baseFrame = Bmc.defaultAvatar.currentFrame;
var frameRate = 60;
var duration = 4.0;
var frameCount = (frameRate * duration).asInteger;
var entries;
var clip;

if(baseFrame.isNil) {
    Error("No live Ishidomaru frame has arrived yet").throw;
};

entries = Array.fill(frameCount, { |index|
    var time = index / frameRate;
    var pose = baseFrame.pose.copy;
    var hips = pose.at(\Hips).asArray;
    var phase = time * 2pi * 0.25;

    hips[0] = hips[0] + (sin(phase) * 0.10);
    pose.put(\Hips, hips);

    [
        time,
        BmcFrame(
            "Ishidomaru",
            "algorithmic-sway",
            index,
            time,
            pose
        ).asOSCMessages
    ]
});

clip = BmcAnimationClip(entries, (
    generator: \sine,
    bodyPart: \Hips,
    duration: duration,
    frameRate: frameRate
));

Bmc.library.add(\algorithmicSway, clip);
Bmc.saveClipScd(\algorithmicSway);
)

```

Disable VMC output in XR-Animator temporarily so the live stream does not compete with playback. Then play the generated clip:

```

Bmc.loop(true);
Bmc.play(\algorithmicSway);

```

Ishidomaru should repeat the slow sideways motion. Stop it with:

```

Bmc.stopPlayback;
Bmc.loop(false);

```

Re-enable XR-Animator VMC output before the recording exercise.

## Record a clip

With the live XR-Animator animation working, start a named recording:

```

Bmc.record(\take1, "Ishidomaru", "getting-started-xr");

```

Move for several seconds, then stop:

```

~take1 = Bmc.stopRecording;

```

Inspect the result:

```
~take1.size;  
~take1.duration;  
Bmc.listClips;  
BmcClipLibrary.defaultDirectory;
```

**take1** is retained in memory and, because SCD is the default recording format, is also saved as **take1.scd** in **BmcClipLibrary.defaultDirectory**.

If the returned clip has zero frames, verify that the Bmc monitor's **received** count was increasing while recording and that the encoder used the same avatar and source strings shown above.

## Play back a clip

Disable XR-Animator VMC output so that live data does not compete with the recording. Play the clip:

```
Bmc.play(\take1);
```

Try the basic transport controls:

```
Bmc.pause;  
Bmc.resume;  
Bmc.stopPlayback;
```

Play at half speed and loop:

```
Bmc.rate(0.5);  
Bmc.loop(true);  
Bmc.play(\take1);
```

Return to the defaults afterward:

```
Bmc.stopPlayback;  
Bmc.rate(1.0);  
Bmc.loop(false);
```

To prove that the disk copy can be restored after recompilation, recompile the class library and load it explicitly:

```
Bmc.loadClipScd(  
    BmcClipLibrary.defaultDirectory +/+ "take1.scd",  
    \take1  
);  
Bmc.play(\take1);
```

## Combine two clips on one avatar

This exercise uses **take1** as the base motion and takes both arms from a second recording.

Re-enable XR-Animator VMC output and record a second take containing distinctive arm movement:

```
Bmc.record(\armTake, "Ishidomaru", "getting-started-xr");
```

Move the arms for several seconds, then stop:

```
Bmc.stopRecording;
```

Create a new clip whose timing and lower body come from **take1**, while its left and right arms come from **armTake**:

```
Bmc.combineClips(  
    \take1,  
    \armTake,  
    \arms,  
    \combinedTake  
);  
Bmc.saveClipScd(\combinedTake);
```

Disable XR-Animator VMC output and play the result:

```
Bmc.play(\combinedTake);
```

The combined clip lasts as long as **take1**. Arm data is replaced only for the number of frames available in both clips; any remaining base frames stay unchanged.

## Drive two avatars from a clip and XR-Animator

This exercise uses the uniform two-avatar project. Ishidomaru follows the live XR-Animator source while Mother plays a saved clip independently.

### 1. Change the Godot project

Stop the current Godot scene. Open and run:

**AppsAndCode/GodotProjects/Seed\_4\_Mother\_Ishidomaru\_E/project.godot**

This project uses:

| Avatar     | VMC port |
|------------|----------|
| Mother     | 39539    |
| Ishidomaru | 39540    |

### 2. Restart the decoder for both avatars

Stop the existing decoder with **Control-C**, then run:

```
python3 PipelineApplications/Bunraku0SCDecoder.py \  
  --listen-port 39538 \  
  --allow-target-port 39539 \  
  --allow-target-port 39540 \  
  --verbose
```

### 3. Register the two Bmc avatar outputs

Evaluate:

```
(  
  ~ishidomaru = Bmc.avatar(\Ishidomaru);  
  ~ishidomaru.vmcPort_(39540);  
  
  ~mother = Bmc.avatar(\Mother);  
  if(~mother.isNil) {  
    ~mother = Bmc.addAvatar(\Mother, "Mother");  
  };  
  ~mother.vmcPort_(39539);  
)
```

### 4. Play a clip on Mother

Create an independent player for Mother. This is separate from Bmc's single convenience player, allowing live Ishidomaru input and Mother playback to occur at the same time:

```
~motherPlayer = BmcClipPlayer(Bmc.clip(\combinedTake), ~mother);  
~motherPlayer.loop_(true);  
~motherPlayer.play;
```

### 5. Restore live Ishidomaru input

Re-enable XR-Animator VMC output. The encoder should still use:

**Avatar:** Ishidomaru  
**Source:** getting-started-xr

Ishidomaru should now follow the webcam while Mother repeats **combinedTake**.

Stop Mother's independent playback with:

```
~motherPlayer.stop;
```

This example establishes two independent motion paths. Session definitions and the figure-composition system will later provide a higher-level way to save and coordinate several such assignments.

## Shut down

Stop playback and Bmc:

```
if(~motherPlayer.notNull) { ~motherPlayer.stop };  
Bmc.stopPlayback;  
Bmc.stop;
```

Then disable XR-Animator VMC output, stop the encoder and decoder with **Control-C**, and stop the Godot scene.